Python Programming Fundamentals

Lists and Loops

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1. Iterating Over Lists

The `for` loop is the natural way to process every item in a list.

```
temperatures = [15.2, 18.7, 12.4, 22.1, 19.8]

# Print each temperature
for temp in temperatures:
    print(f"{temp:.1f}C")

# Accumulation — find total
total = 0
for temp in temperatures:
    total += temp
average = total / len(temperatures)
print(f"Average: {average:.1f}C")
```

1. Iterating Over Lists

1. Iterating Over Lists

The `for` loop is the natural way to process every item in a list.

```
temperatures = [15.2, 18.7, 12.4, 22.1, 19.8]

# Print each temperature
for temp in temperatures:
    print(f"{temp:.1f}C")

# Accumulation — find total
total = 0
for temp in temperatures:
    total += temp
average = total / len(temperatures)
print(f"Average: {average:.1f}C")
```

Python also has built-in functions for common operations:

1. Iterating Over Lists

```
print(sum(temperatures))        # total
print(min(temperatures))        # lowest
print(max(temperatures))        # highest
```

Outline

2. enumerate() with Lists

`enumerate()` gives both the **index** and the **value**.

```
students = ["Alice", "Bob", "Carol", "Dave"]

for i, student in enumerate(students):
    print(f"Student {i}: {student}")

# Start numbering from 1
for i, student in enumerate(students, start=1):
    print(f"{i}. {student}")
```

2. enumerate() with Lists

2. enumerate() with Lists

`enumerate()` gives both the **index** and the **value**.

```
students = ["Alice", "Bob", "Carol", "Dave"]

for i, student in enumerate(students):
    print(f"Student {i}: {student}")

# Start numbering from 1
for i, student in enumerate(students, start=1):
    print(f"{i}. {student}")
```

Updating items using index:

```
scores = [72, 85, 91, 64, 78]

# Add 5 bonus points to each score
for i in range(len(scores)):
```

2. enumerate() with Lists

```
    scores[i] += 5

print(scores)   # [77, 90, 96, 69, 83]
```

Outline

3. Finding Min and Max Manually

Useful to understand the algorithm, even though Python has built-ins.

```
def find_min(numbers):
    """Return the smallest number in a non-empty list."""
    smallest = numbers[0]
    for num in numbers[1:]:
        if num < smallest:
            smallest = num
    return smallest

def find_max_index(numbers):
    """Return the INDEX of the largest number."""
    max_idx = 0
    for i in range(1, len(numbers)):
        if numbers[i] > numbers[max_idx]:
            max_idx = i
```

3. Finding Min and Max Manually

```
    return max_idx

data = [34, 12, 78, 45, 23, 91, 56]
print(find_min(data))                    # 12
print(find_max_index(data))              # 5
print(data[find_max_index(data)])        # 91
```

Outline

4. Filtering a List

Manual filter — build a new list with only matching items:

```
scores = [45, 72, 88, 53, 91, 64, 78, 39, 85]

passing = []
for score in scores:
    if score >= 60:
        passing.append(score)

print(f"Passing scores: {passing}")
print(f"Pass rate: {len(passing)/len(scores):.0%}")
```

4. Filtering a List

4. Filtering a List

Manual filter — build a new list with only matching items:

```
scores = [45, 72, 88, 53, 91, 64, 78, 39, 85]

passing = []
for score in scores:
    if score >= 60:
        passing.append(score)

print(f"Passing scores: {passing}")
print(f"Pass rate: {len(passing)/len(scores):.0%}")
```

Using the built-in `filter()` function:

```
passing = list(filter(lambda s: s >= 60, scores))
```

Or a list comprehension (most Pythonic):

4. Filtering a List

```
passing = [s for s in scores if s >= 60]
```

Outline

5. List Comprehensions

A compact way to build a new list from an existing one.

```
# Traditional for loop
squares = []
for x in range(1, 8):
    squares.append(x ** 2)

# Equivalent list comprehension
squares = [x ** 2 for x in range(1, 8)]
print(squares)   # [1, 4, 9, 16, 25, 36, 49]
```

5. List Comprehensions

5. List Comprehensions

A compact way to build a new list from an existing one.

```
# Traditional for loop
squares = []
for x in range(1, 8):
    squares.append(x ** 2)

# Equivalent list comprehension
squares = [x ** 2 for x in range(1, 8)]
print(squares)   # [1, 4, 9, 16, 25, 36, 49]
```

With a condition:

```
numbers = [-3, 5, -1, 8, -4, 2, 7]
positives = [n for n in numbers if n > 0]
print(positives)   # [5, 8, 2, 7]
```

5. List Comprehensions

```
# Transform strings
names = ["  alice  ", "BOB", "Carol"]
cleaned = [n.strip().title() for n in names]
print(cleaned)   # ['Alice', 'Bob', 'Carol']
```

Outline

6. While Loops with Lists

```
# Process items until the list is empty
queue = ["Alice", "Bob", "Carol"]

while queue:              # True while list is non-empty
    next_person = queue.pop(0)
    print(f"Serving: {next_person}")

print("All done!")
```

6. While Loops with Lists

6. While Loops with Lists

```
# Process items until the list is empty
queue = ["Alice", "Bob", "Carol"]

while queue:              # True while list is non-empty
    next_person = queue.pop(0)
    print(f"Serving: {next_person}")

print("All done!")
```

```
# Collect until sentinel
numbers = []
print("Enter numbers (0 to stop):")
while True:
    value = float(input("> "))
    if value == 0:
        break
```

6. While Loops with Lists

```
        numbers.append(value)

if numbers:
    print(f"Count:   {len(numbers)}")
    print(f"Sum:     {sum(numbers):.2f}")
    print(f"Average: {sum(numbers)/len(numbers):.2f}")
```

Outline

7. Practical — Statistics on a List

```
def analyse(data):
    """Return a dict of statistics for a numeric list."""
    if not data:
        return {}
    sorted_data = sorted(data)
    n = len(sorted_data)
    return {
        "count": n,
        "sum":   sum(sorted_data),
        "min":   sorted_data[0],
        "max":   sorted_data[-1],
        "mean":  sum(sorted_data) / n,
        "median": sorted_data[n // 2] if n % 2 != 0
                  else (sorted_data[n//2-1] + sorted_data[n//2]) / 2,
    }
```

7. Practical — Statistics on a List

```
scores = [78, 92, 85, 71, 88, 95, 64, 82, 90, 77]
stats = analyse(scores)
for key, value in stats.items():
    if isinstance(value, float):
        print(f"  {key:<8}: {value:.2f}")
    else:
        print(f"  {key:<8}: {value}")
```

Thanks for Watching - Any questions?